

Cognitive Robotics Lab
Lab 9: Create a path planning (SW) using grid generated
Srikar Ganesh 22BAI1080
13-10-2025

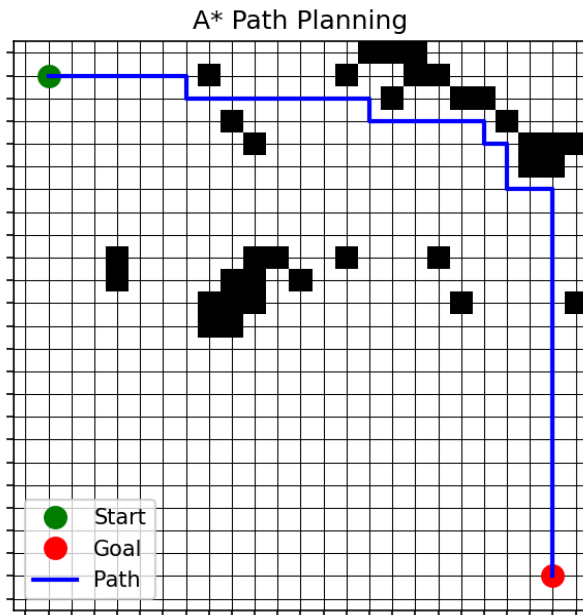
Aim:

To design and implement a path planning algorithm that allows a robot to navigate from a start point to a goal point within a grid-based environment while avoiding obstacles. The experiment demonstrates how autonomous systems can compute optimal or feasible paths using algorithms such as A* on a generated occupancy grid.

Components:

1. Computer or laptop
2. Python 3.x
3. Matplotlib library for visualization
4. NumPy library for numerical computation
5. Heapq or queue module for algorithm implementation
6. PIL (Python Imaging Library) for grid generation and display (optional)
7. Jupyter Notebook, VS Code, or PyCharm for development
8. Grid map representation using a 2D array or matrix
9. Path planning algorithm such as A*, Dijkstra, or BFS
10. Visualization module for displaying the grid, obstacles, and final path

Output



Source Code:

```
import json
import heapq
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors

def load_grid_map(file_path):
    """Loads the grid map from a JSON file."""
    try:
        with open(file_path, 'r') as f:
            return json.load(f)
    except FileNotFoundError:
        print(f"Error: The file '{file_path}' was not found.")
        return None
    except json.JSONDecodeError:
        print(f"Error: Could not decode JSON from the file '{file_path}'.")
        return None

def heuristic(a, b):
    """Calculates the Manhattan distance heuristic between two points."""
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def a_star_search(grid, start, goal):
    """
    Finds the shortest path from start to goal using the A* algorithm.
    """
```

```

rows, cols = len(grid), len(grid[0])
# Check if start or goal are obstacles
if grid[start[0]][start[1]] == 1 or grid[goal[0]][goal[1]] == 1:
    print("Start or Goal is on an obstacle.")
    return None

# The set of discovered nodes that may need to be (re-)expanded.
# Initially, only the start node is known.
# Implemented as a priority queue.
open_set = []
heapq.heappush(open_set, (0, start))

# For node n, came_from[n] is the node immediately preceding it on the cheapest path from
start to n.
came_from = {}

# For node n, g_score[n] is the cost of the cheapest path from start to n currently known.
g_score = { (r, c): float('inf') for r in range(rows) for c in range(cols) }
g_score[start] = 0

# For node n, f_score[n] := g_score[n] + h(n). f_score[n] represents our current best guess as to
# how short a path from start to finish can be if it goes through n.
f_score = { (r, c): float('inf') for r in range(rows) for c in range(cols) }
f_score[start] = heuristic(start, goal)

while open_set:
    # Get the node in open_set having the lowest f_score value
    _, current = heapq.heappop(open_set)

    if current == goal:
        return reconstruct_path(came_from, current)

    # Explore neighbors
    for dr, dc in [(0, 1), (0, -1), (1, 0), (-1, 0)]: # 4-directional movement
        neighbor = (current[0] + dr, current[1] + dc)

        # Check if neighbor is valid
        if 0 <= neighbor[0] < rows and 0 <= neighbor[1] < cols and grid[neighbor[0]][neighbor[1]]
== 0:
            # d(current,neighbor) is the weight of the edge from current to neighbor
            # tentative_g_score is the distance from start to the neighbor through current
            tentative_g_score = g_score[current] + 1
            if tentative_g_score < g_score[neighbor]:
                # This path to neighbor is better than any previous one. Record it!
                came_from[neighbor] = current
                g_score[neighbor] = tentative_g_score

```

```

        f_score[neighbor] = g_score[neighbor] + heuristic(neighbor, goal)
        if (f_score[neighbor], neighbor) not in open_set:
            heapq.heappush(open_set, (f_score[neighbor], neighbor))

    return None # No path found

def reconstruct_path(came_from, current):
    """Reconstructs the path from the came_from map."""
    total_path = [current]
    while current in came_from:
        current = came_from[current]
        total_path.append(current)
    return total_path[::-1] # Return reversed path

def visualize_path(grid, path, start, goal, output_filename=None):
    """Visualizes the grid, obstacles, and the path, and optionally saves it."""
    # Create a colormap: 0=white (free), 1=black (obstacle)
    cmap = mcolors.ListedColormap(['white', 'black'])
    bounds = [-0.5, 0.5, 1.5]
    norm = mcolors.BoundaryNorm(bounds, cmap.N)

    fig, ax = plt.subplots()
    ax.imshow(grid, cmap=cmap, norm=norm)

    # Draw gridlines
    ax.grid(which='major', axis='both', linestyle='-', color='k', linewidth=0.5)
    ax.set_xticks(range(len(grid[0])))
    ax.set_yticks(range(len(grid)))
    ax.set_xticklabels([])
    ax.set_yticklabels([])

    # Plot start and goal points
    ax.plot(start[1], start[0], 'go', markersize=10, label='Start') # Green circle
    ax.plot(goal[1], goal[0], 'ro', markersize=10, label='Goal') # Red circle

    # Plot the path
    if path:
        path_rows, path_cols = zip(*path)
        ax.plot(path_cols, path_rows, 'b-', linewidth=2, label='Path') # Blue line

    ax.legend()
    plt.title("A* Path Planning")

    # Save the figure if a filename is provided
    if output_filename:
        plt.savefig(output_filename, bbox_inches='tight', dpi=150)

```

```

    print(f"Path visualization saved to {output_filename}")

plt.show()

if __name__ == "__main__":
    # The JSON file is in the same directory as the script
    grid_map_file = 'grid_map (1).json'
    output_image_file = '22BA11080_PathPlanning_Output.png'
    grid = load_grid_map(grid_map_file)

    if grid:
        # Define start and goal points (row, col)
        # Make sure these points are not obstacles (value 0)
        start_point = (1, 1)
        goal_point = (23, 23)

        # Find the path
        path = a_star_search(grid, start_point, goal_point)

        if path:
            print(f"Path found from {start_point} to {goal_point}.")
            # print("Path:", path)
        else:
            print(f"No path could be found from {start_point} to {goal_point}.")

        # Visualize the result
        visualize_path(grid, path, start_point, goal_point, output_image_file)

```

Explanation:

The code implements a grid-based path planning system using the A* algorithm. It begins by loading a JSON file containing a map represented as a 2D grid, where 0 denotes free space and 1 denotes an obstacle. The A* algorithm combines the actual distance from the start node and an estimated heuristic distance (Manhattan distance) to the goal, ensuring both optimality and efficiency. It explores valid neighboring cells while maintaining a priority queue of possible paths, selecting the one with the lowest total estimated cost at each step. Once the goal is reached, the program reconstructs the path and visualizes it using matplotlib, highlighting the start, goal, obstacles, and the final computed route.

Result:

The experiment successfully demonstrates how the A* algorithm can be applied for efficient path planning in a grid-based environment. The robot (or simulated agent) effectively finds the shortest path from the start to the goal point while avoiding obstacles. This lab reinforces the importance of heuristic-based search in robotics and autonomous navigation, showing how such algorithms enable intelligent decision-making in real-world robotic systems.